
Data Structures

(Stack)

Introduction

Faculty Name: Naeem Ahmed (NMA)

Room: UB80601

Consultation Hours:

Sunday: 9:30 AM - 11:00 AM

Monday: 11:00 AM-12:30 PM

Wednesday: 11:00 AM-2:00 PM



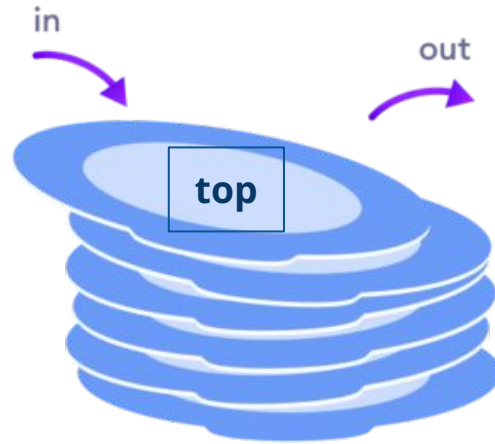
Stack



Stack Data Structure

- Store information, and access the more recently stored information first
- Maintains Last In First Out (LIFO) principle
- Example: Stack of plates, deck of cards, piles of books, piles of money etc

Stack Data Structure



Stack is a restricted ordered sequence in which we can only add to and remove from one end — the top of the stack

Stack Data Structure

You are not, strictly speaking, allowed to

- Add to the middle of the stack
- Remove a plate from the middle of the stack
- The only plate that can be taken out of the stack is the most recently added one
- A stack is thus a "last in, first out" (LIFO) data structure.

Stack Data Structure



STACK DATA STRUCTURE

Stack Operations

Three basic stack operations are:

push(obj): adds obj to the top of the stack ("overflow" error if the stack has fixed capacity, and is full)

pop: removes and returns the item from the top of the stack ("underflow" error if the stack is empty)

peek: returns the item that is on the top of the stack, but does not remove it ("underflow" error if the stack is empty)

Stack Applications : Call Stacks

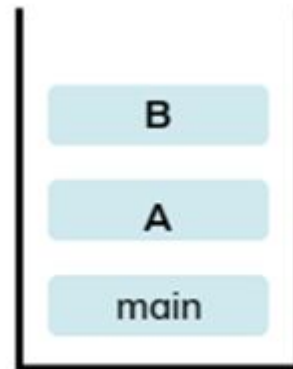
- Function calls are stored as stack records
- Whenever your C program calls a function, the C compiler creates a so-called stack record that contains important information about that function, such as storage for its parameters, its return value, and its local variables.
- The C compiler pushes this stack record onto something called a call stack.
- The call stack is organized so that the stack record of the most recently called function is on top of the stack, and the stack record of the least recently called function, which in the case of C is main, is at the bottom of the stack.

Stack Applications : Call Stacks

- For example, if the main calls A, which in turn calls B, then the call stack would be:

Corresponding code:

```
def B():  
    print("B")  
def A():  
    B()  
    print("A")  
def main():  
    A()  
    print("Main")  
if __name__ == "__main__":  
    main()
```

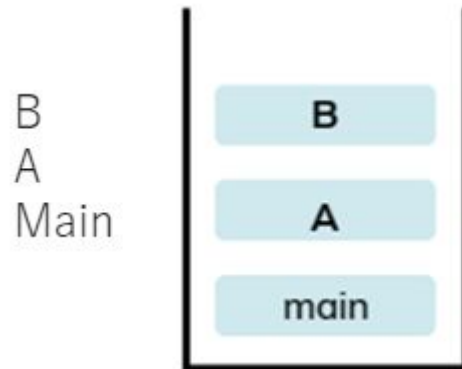


Stack Applications : Call Stacks

- For example, if the main calls A, which in turn calls B, then the call stack would be:

Corresponding code:

```
def B():  
    print("B")  
def A():  
    B()  
    print("A")  
def main():  
    A()  
    print("Main")  
if __name__ == "__main__":  
    main()
```



Stack Applications : Reverse

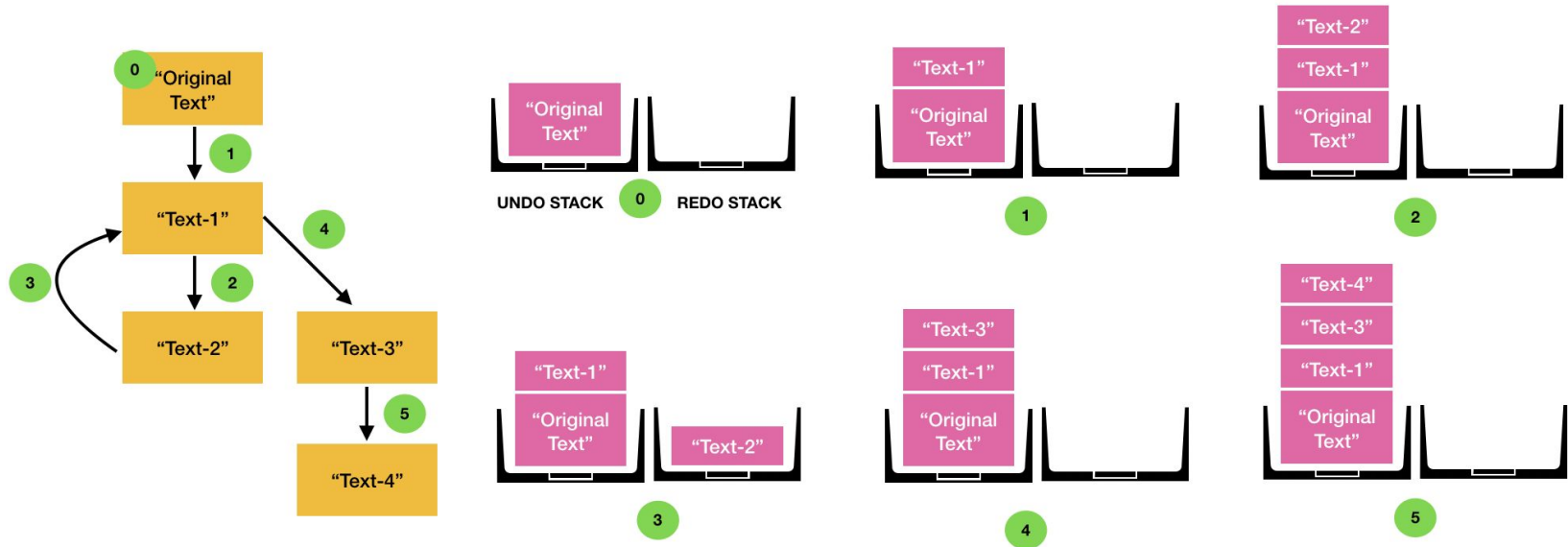
- The simplest application of a stack is to reverse a word. You push a given word to stack – letter by letter – and then pop letters from the stack.

```
begin with an empty stack and an input stream.  
while there is more characters to read, do:  
    read the next input character;  
    push it onto the stack;  
end while;  
while the stack is not empty, do:  
    c = pop the stack;  
    print c;  
end while;
```

Stack Applications : Undo

- An "undo" mechanism in text editors
- Accomplished by keeping all text changes in a stack
- Popping the stack is equivalent to "undoing" the last action
- Similar: going back pages in a browser using the back button

Stack Applications : Undo

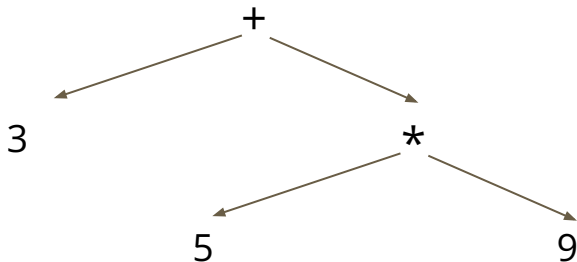


This is just for visualization purposes, no need to memorize

Stack Applications : Expression Evaluation

- When an arithmetic expression is presented in the postfix form, you can use a stack to evaluate it to get the final value.
- For example: the expression $3 + 5 * 9$ (which is in the usual infix form) can be written as $3\ 5\ 9\ *\ +$ in the postfix.

Postfix: left subtree -> right subtree -> root



Stack Applications : Expression Evaluation

- Postfix form removes all parentheses and thus all implicit precedence rules
- **Task:** write the postfix form corresponding to the following in-fix form:
 $((3 + 2) * 4) / (5 - 1)$

Stack Applications : Expression Evaluation

- Postfix form removes all parentheses and thus all implicit precedence rules
- **Task:** write the postfix form corresponding to the following in-fix form:

$((3 + 2) * 4) / (5 - 1)$

Answer: $3\ 2\ +\ 4\ *\ 5\ 1\ -\ /\$

Stack Applications : Expression Evaluation

You can now design a calculator for expressions in postfix form using a stack.

Stack Applications : Expression Evaluation

```
begin with an empty stack and an input stream (for the expression).
while there is more input to read, do:
    read the next input symbol;
    if it's an operand,
        then push it onto the stack;
    if it's an operator
        then pop two operands from the stack;
        perform the operation on the operands;
        push the result;
end while;
// the answer of the expression is waiting for you in the stack:
pop the answer;
```

Stack Applications : Expression Evaluation

Let's apply this algorithm to to evaluate the postfix expression $3\ 2\ +\ 4\ *\ 5\ 1\ -\ /$ using a stack:

Stack Applications : Expression Evaluation

Stack	Expression
 --- (empty)	3 2 + 4 * 5 1 - /
3 ---	2 + 4 * 5 1 - /
2 3 ---	+ 4 * 5 1 - /
5 ---	4 * 5 1 - /
4 5 ---	* 5 1 - /
20 ---	5 1 - /
5 20 ---	1 - /

Stack Applications : Expression Evaluation

```
| 1 |  
| 5 |  
| 20 |  
---
```

- /

```
| 4 |  
| 20 |  
---
```

/

```
| 5 |  
---
```

(finished. the result is on top of the stack)

Stack Applications : Expression Evaluation

Convert $(1+2)*(4-3) + 1$ into the postfix format and evaluate the postfix expression using stack

Stack Applications : Parentheses Matching

- In many editors, it is common to want to know which left parenthesis (or brace or bracket, etc) will be matched when we type a right parenthesis
- Clearly we want the most recent left parenthesis.
- Therefore, the editor can keep track of parentheses by pushing them onto a stack, and then popping them off as each right parenthesis is encountered.
- We often have expressions involving "()[]{}" that require that the different types of parentheses are balanced.

Stack Applications : Parentheses Matching

The following are properly balanced:

- $(a (b + c) + d)$
- $[(a\ b)(c\ d)]$
- $([a\{x\ y\}b])$

The following are not properly balanced:

Stack Applications : Parentheses Matching

The Algorithm to follow

```
begin with an empty stack and an input stream (for the expression).
while there is more input to read, do:
    read the next input character;
    if it's an opening parenthesis/brace/bracket "(" or "{" or "["
        then push it onto the stack;
    if it's a closing parenthesis/brace/bracket ")" or "}" or "]"
        then pop the opening symbol from stack;
        compare the closing with opening symbol;
        if it matches
            then continue with next input character;
        if it does not match
            then return false;
end while;
// all matched, so return true
return true;
```

Stack Applications : Backtracking

This is a process when you need to access the most recent data element in a series of elements. Think of a labyrinth or maze - how do you find a way from an entrance to an exit?



Stack Applications : Backtracking

- Once you reach a dead end, you must backtrack. But backtrack to where? to the previous choice point.
- Therefore, at each choice point you store on a stack all possible choices.
- Then backtracking simply means popping a next choice from the stack.

Stack Applications : Language Processing

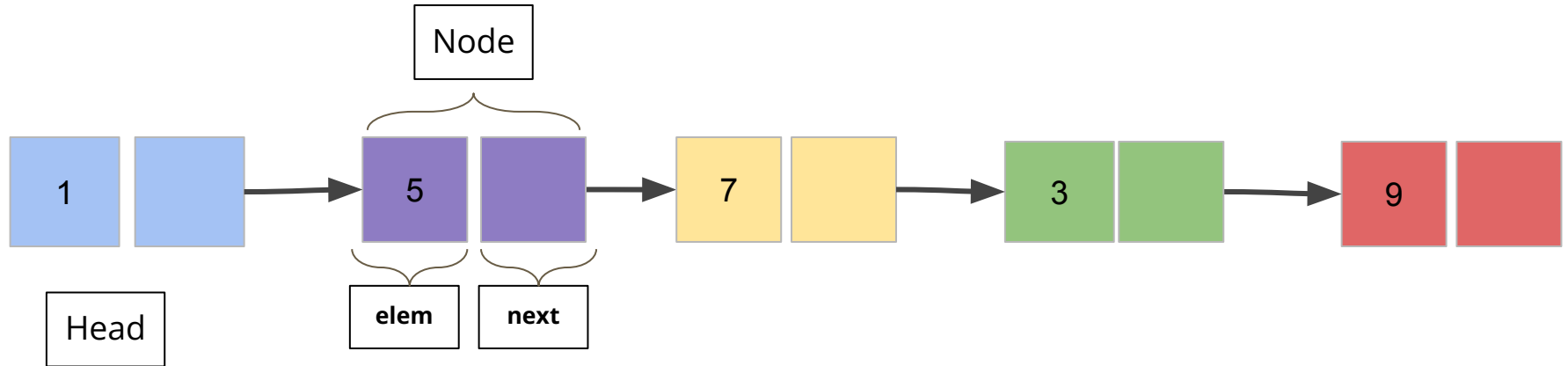
- Space for parameters and local variables is created internally using a stack (activation records).
- The compiler's syntax check for matching braces is implemented by using stack.
- Support for recursion

Stack Implementation

— Using Linked List —

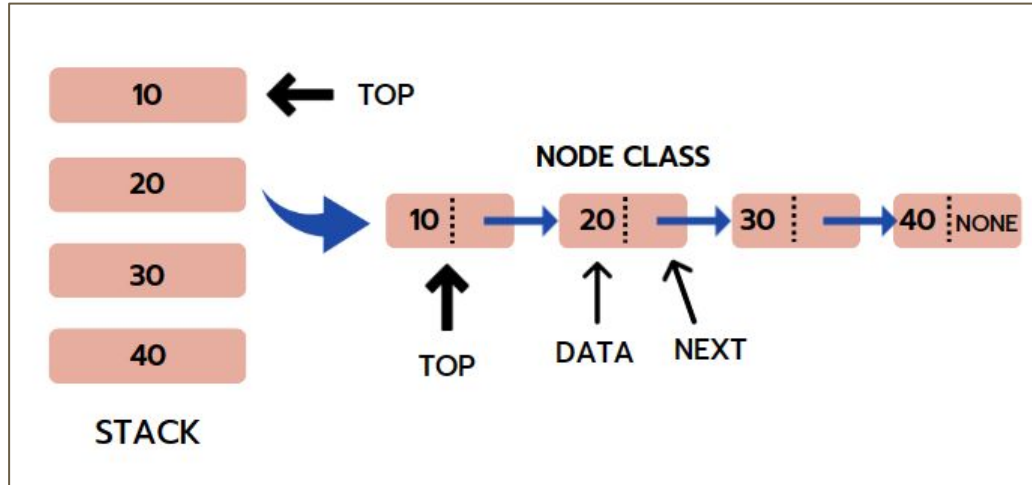
Linked List

We'll use the non dummy-headed singly linear linked list



```
class Node:
    def __init__(self, element, next):
        self.element = element
        self.next = next
```

Use of Linked List in Stack implementation



- The Top element is the Head of Linked List
- New element is added to the Head/Top (ie. push means inserting to the head position)
- Any element removal happens from the head (ie. pop means removing the head)

Stack Operations

1. Push
2. Pop
3. Peek

Push operation implementation

```
class Stack:
    def __init__(self):
        self.top = None

    def push(self, elem):
        if self.top == None: #Stack is empty
            self.top = Node(elem, None)
        else:
            newNode = Node(elem, None)
            newNode.next = self.top
            self.top = newNode
```

Push operation implementation

Adding a new node in the Stack is termed a push operation. Push operation on stack implementation using linked-list involves several steps:

- Create a node first and allocate memory to it.
- If the stack is empty, then the node is pushed as the first node of the linked list and defined as top. This operation assigns a value to the data part of the node and gives None to the address part of the node.
- If some nodes are already there in the linked list, then we have to add a new node at the beginning of the linked list so that we do not violate Stack's property. For this, assign the previous top to the next address field of the new node and the new node will be the starting node or the current top of the list.

Pop operation implementation

```
def pop(self):  
    if self.top == None:  
        return None #Stack Underflow  
    else:  
        popped = self.top  
        self.top = self.top.next  
        popped.next = None  
        return popped.element
```

Pop operation implementation

Deleting a node from the Stack is known as a pop operation. Pop operation involves the following steps:

- In Stack, the node is removed from the top of the linked list. Therefore, we must delete the value stored in the head/top pointer, and the node must get free. The following link node then becomes the head/top node.
- An underflow condition will occur when we try to pop a node when the Stack is empty.

Peek operation implementation

In the peek operation, we simply return the data stored in the head/top of the Linked List based Stack. An underflow condition will occur when we try to peek at a value when the Stack is empty.

```
def peek(self):  
    if self.top == None:  
        return None #Stack Underflow  
    else:  
        return self.top.element
```

Queue

Queue



Queue Data Structure

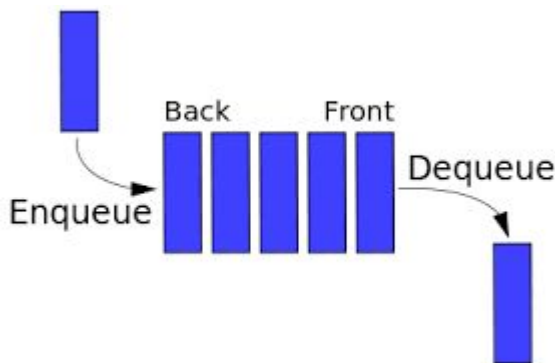
- A queue is a new data structure that follows **First In First Out (FIFO)** while inserting and retrieving data.
- If you are waiting in line to buy a bus ticket, the counter will serve the customer who has come first and a new customer will wait behind the last person waiting in line.

Queue Data Structure in computers is resource management

- For example, think of an office scenario where 10 people use the same printer. So if everyone requests to print simultaneously, the printer will print the first request and store the other requests in a queue. While executing any printing operation if a new print request comes it will store it in the same queue and execute the command in the First In First Out manner.
- In addition, when we execute 100 programs in our 8-core CPU, the processor also maintains a queue for command execution (Though the scenario is a bit more complex which you will learn in Operating Systems).

Queue Operations

- **Enqueue:** It stores a **new object** in the queue **at the end**. If the queue is full, this operation throws a QueueOverflow exception.
- **Dequeue:** It **removes** the **first object** from the queue. If the queue is empty, this operation throws a QueueUnderflow exception.
- **Peek:** This operation **shows** the **first item** in the queue. This operation **does not remove** any items from the queue. If the queue is empty, this operation throws a QueueUnderflow exception.



Queue Implementation

— Using Linked List —

Queue Implementation

As we can see, to maintain the queue, we need two pointers, one for item retrieval and one for item insertion. As a result, we will maintain a front and a back for a queue. The idea is whenever we dequeue an item or peek at an item, we will use the front pointer and to enqueue an item we will use the back pointer.

Queue Implementation

The Node class:

```
class Node:  
    def __init__(self, elem, next):  
        self.elem = elem  
        self.next = next
```

Enqueue

```
1 class Queue:
2
3     def __init__(self):
4         self.front = None
5         self.back = None
6
7     def enqueue(self, elem):
8         if self.front == None: # This means first item
9             self.front = Node(elem, None)
10            self.back = self.front # Initially front and back are same
11        else:
12            n = Node(elem, None)
13            self.back.next = n # Inserting at the last
14            self.back = self.back.next # Moving back at the end
```

Dequeue

```
def dequeue(self):  
    if self.front == None: # For empty Queue  
        print("Queue Underflow Exception")  
    else:  
        dequeued_item = self.front.elem  
        self.front = self.front.next # Moving the front pointer  
        return dequeued_item # Returning the dequeued item
```

Peek

```
def peek(self):  
    if self.front == None: # For empty Queue  
        print("Queue Underflow Exception")  
    else:  
        return self.front.elem # Returning the front item
```

To be noted

Note that here, we are maintaining a front (for dequeue and peek) and a back (for enqueue). As the size of the linked list is not fixed, we are not maintaining the QueueOverflow exception in the given code.

Queue Implementation

Using Array

Queue Implementation

- If we want to create a queue with fixed size then array based queue is a good approach.
- Here we will also use a front index and a back index. Initially both will start from the same index.
- After that, after every enqueue the back index will move one index further.
- After every dequeue the front index will move one index further.
- To keep indexing convenient we will use a circular array to implement the queue.
- To determine the queue overflow exception, we will maintain a size variable.
- Array based queue are also known as buffer. Buffer is used in many applications such as web server request management.

Enqueue

```
1 class ArrayQueue:
2     def __init__(self):
3         self.queue = [None] * 10 # Queue with size 10
4         self.front = 0 # Initializing at index 0
5         self.back = 0 # Initializing at index 0
6         self.size = 0 # no elements
7
8     def enqueue(self, elem):
9         if self.size == len(self.queue):
10             print("Queue Overflow")
11         else:
12             self.queue[self.back] = elem
13             self.back = (self.back + 1) % len(self.queue)
14             self.size += 1
15
```

Dequeue

```
15
16 def dequeue(self):
17     if self.size == 0:
18         print("Queue Underflow")
19     else:
20         dequeued_item = self.queue[self.front]
21         self.queue[self.front] = None
22         self.front = (self.front + 1) % len(self.queue)
23         self.size -= 1
24         return dequeued_item
25
```

Peek

```
26 def peek(self):  
27     if self.size == 0:  
28         print("Queue Underflow")  
29     else:  
30         return self.queue[self.front]
```